САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Лабораторная работа №1 «Реализация REST API на основе boilerplate»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать полноценное REST API для выбранного приложения (выбрал свой вариант, мини-фото-соцсеть) с использованием Node.js, Express, TypeORM и SQLite, следуя результатам, полученным в ДЗ1 (проектирование БД) и ДЗ2 (проектирование API). API должно поддерживать: аутентификацию (JWT), создание постов с фотографиями (оригинал + превью), комментарии (текст и опциональное фото), лайки, подписки, избранное, уведомления, а также удаление ресурсов с очисткой файлов.

Выбранный вариант:
Фото-соцсеть со следующими возможностями:

- загрузка фото в высоком разрешении для просмотра;
- комментарии с прикреплением фото (сжатие до примерно 4 Мп без потери качества цвета, то есть только по разрешению);
- лента новостей на основе подписок;
- уведомления о лайках, комментариях, подписках.

Ход работы

1. Настройка окружения и структуры проекта

- Установлен Node.js v24, npm, TypeScript.
- Инициализирован проект, установлены зависимости: express, typeorm, sqlite3, reflect-metadata, multer, sharp, bcryptjs, jsonwebtoken, а также типы для разработки.
- Создана структура папок: src/entity, src/controller, src/routes, src/middleware, src/utils, uploads/.

2. Проектирование базы данных и реализация сущностей

На основе ER-диаграммы, разработанной в ДЗ1, созданы TypeORM-модели (сущности) в папке src/entity:

- User – пользователи (id, username, email, passwordHash, avatarPath, createdAt)
- Post – посты с фото (id, userId, originalPhotoPath, previewPhotoPath, description, createdAt)
- Like – лайки (id, userId, postId, createdAt) с уникальным ограничением (userId, postId)

- Comment – комментарии (id, userId, postId, text, photoPath, createdAt)
- Follow – подписки (id, followerId, followingId, createdAt) с уникальным ограничением (followerId, followingId)
- Favorite – избранное (id, userId, postId, createdAt) с уникальным ограничением (userId, postId)
- Notification – уведомления (id, userId, type, relatedUserId, relatedPostId, isRead, createdAt)

Все связи “один ко многим” реализованы через декораторы @ManyToOne и @OneToMany с каскадным удалением (onDelete: "CASCADE"). Для SQLite в полях с nullable: true явно указан тип (varchar, integer), чтобы избежать ошибок при миграции.

3. Реализация API эндпоинтов (REST)

В папке src/controller созданы контроллеры, каждый из которых содержит статические методы для обработки запросов. В src/routes определены роуты, которые подключаются в index.ts. Используется middleware authenticate для проверки JWT-токена (кроме эндпоинтов регистрации и логина).

Всего реализовано 19 эндпоинтов:

Метод	**Эндпоинт**	**Описание**
POST	/api/auth/register	Регистрация
POST	/api/auth/login	Логин, выдача JWT
GET	/api/users/profile	Получение профиля
POST	/api/users/avatar	Загрузка аватара
POST	/api/users/follow/:followingId	Подписаться
DELETE	/api/users/follow/:followingId	Отписаться
POST	/api/posts	Создать пост (фото + описание)
GET	/api/posts/feed	Лента подписок
GET	/api/posts/:id	Получить пост с комментариями
DELETE	/api/posts/:id	Удалить свой пост
POST	/api/posts/:id/like	Поставить лайк
DELETE	/api/posts/:id/like	Убрать лайк
POST	/api/posts/:id/favorite	Добавить в избранное
DELETE	/api/posts/:id/favorite	Удалить из избранного
POST	/api/comments/posts/:postId/comments	Добавить комментарий (текст + фото)
DELETE	/api/comments/comments/:id	Удалить комментарий
GET	/api/comments/posts/:postId/comments	Получить комментарии поста

GET	/api/notifications	Получить уведомления
PUT	/api/notifications/:id/read	Отметить уведомление прочитанным

4. Обработка изображений (sharp)

Для загрузки файлов используется multer с временным сохранением в uploads/temp/. Далее sharp выполняет преобразования:

- Аватар: ресайз до 200×200 px, качество JPEG 85%.
- Фото поста: создаётся оригинал (JPEG, качество 95%) и превью (ширина 1200px, качество 85%). Оба файла сохраняются в uploads/posts/.
- Фото комментария: ресайз до ширины 2560px (≈4 Мп) с сохранением пропорций, качество JPEG 95% (чтобы избежать артефактов сжатия цвета). Сохраняется в uploads/comment-photos/.

Пути к файлам сохраняются в соответствующих полях БД. Статическая раздача папки uploads настроена через express.static, что позволяет просматривать оригиналы фото в браузере по URL типа http://localhost:3000/uploads/posts/original_....jpg.

5. Аутентификация и безопасность

Пароли хешируются с помощью bcryptjs (10 раундов). JWT-токен создаётся при логине, содержит userId, срок действия – 7 дней.
Middleware authenticate извлекает токен из заголовка Authorization: Bearer <token>, верифицирует его и добавляет req.userId. Все защищённые эндпоинты используют эту middleware. При удалении поста или комментария проверяется, что текущий пользователь является владельцем, иначе возвращается ошибка 403. Файлы, связанные с удалённым ресурсом, также удаляются с диска с помощью fs.unlink.

6. Тестирование API

Тестирование проводилось с помощью команд curl в PowerShell и Thunder Client (расширение VS Code). Ниже приведены примеры успешных запросов и ответов.

Регистрация пользователя alice:

```
Invoke-RestMethod -Uri http://localhost:3000/api/auth/register -Method Post -Headers @{"Content-Type"="application/json"} -Body '{"username":"alice","email":"alice@example.com","password":"123456"}'
```

Ответ:

{"id":2,"username":"alice","email":"alice@example.com"}

Логин и получение токена:

Invoke-RestMethod -Uri http://localhost:3000/api/auth/login -Method Post -Headers @{"Content-Type"="application/json"} -Body '{"email":"alice@example.com","password":"123456"}'

Ответ:

(содержит <token>)

Создание поста с фото (curl):

curl -X POST http://localhost:3000/api/posts -H "Authorization: Bearer <токен>" -F "photo=@C:\path\photo.jpg" -F "description=Мой первый пост, фоткал на олимпус"

Ответ:

{"id":1,"userId":1,"originalPhotoPath":"/uploads/posts/original_...jpg","previewPhotoPath":"/uploads/posts/preview_...jpg","description":" Мой первый пост, фоткал на олимпус","createdAt":"..."}

Лайк поста:

curl -X POST http://localhost:3000/api/posts/1/like -H "Authorization: Bearer <токен>"

Ответ:

{"message":"Liked"}.

Повторный запрос возвращает {"message":"Already liked"}

Добавление комментария с фото:

curl -X POST http://localhost:3000/api/comments/posts/1/comments -H "Authorization: Bearer <токен>" -F "text=Курлык" -F "photo=@C:\small.jpg"

Ответ:

{"userId":1,"postId":1,"text":"Курлык","photoPath":"/uploads/comment-photos/comment_1_...jpg","id":2,"createdAt":"..."}

Получение ленты подписок:

curl -X GET http://localhost:3000/api/posts/feed -H "Authorization: Bearer <токен>"

Ответ: массив постов от пользователей, на которых подписан текущий пользователь.

Подписка на пользователя:

curl -X POST http://localhost:3000/api/users/follow/2 -H "Authorization: Bearer <токен>"

Ответ: {"message":"Followed"}

Получение уведомлений:

curl -X GET http://localhost:3000/api/notifications -H "Authorization: Bearer <токен>"

Ответ: массив уведомлений с полями type, relatedUserId, isRead и т.д.

Отметка уведомления как прочитанного:

curl -X PUT http://localhost:3000/api/notifications/3/read -H "Authorization: Bearer <токен>"

Ответ: {"message":"Marked as read"}.

При повторном запросе уведомлений поле isRead становится true.

Удаление комментария:

curl -X DELETE http://localhost:3000/api/comments/comments/2 -H "Authorization: Bearer <токен>"

Ответ: {"message":"Comment deleted"}. Файл фото комментария удаляется с диска.

Удаление поста:

curl -X DELETE http://localhost:3000/api/posts/1 -H "Authorization: Bearer <токен>"

Ответ: {"message":"Post deleted"}. Оригинал и превью поста удаляются из uploads/posts

Также протестированы негативные сценарии: попытка удалить чужой пост (403), повторный лайк (409), создание поста без фото (400), подписка на самого себя (400), неверный токен (401), запрос несуществующего ресурса (404). Все ошибки возвращают соответствующее сообщение в формате JSON.

7. Запуск и развёртывание

Для запуска приложения достаточно выполнить две команды:

npm install

npm run dev

Сервер стартует на порту 3000. База данных SQLite создаётся автоматически (файл database.sqlite). Файлы загрузок сохраняются в папку uploads/, которая создаётся при первом запросе. Для остановки сервера используется Ctrl+C.

Вывод

В ходе выполнения ЛР1 полностью разработан бэкенд для фото-соцсети, реализующий перечень требований: аутентификация JWT, создание постов с загрузкой фото (оригинал + превью), лента новостей на основе подписок, лайки, избранное, комментарии с возможностью прикрепления фото (сжатие до ~4 Мп без потери цветности), подписки, уведомления о лайках/комментариях/подписках, а также удаление постов и комментариев с физическим удалением файлов. Все эндпоинты соответствуют спроектированной в ДЗ2 OpenAPI-спецификации и протестированы через curl и Thunder Client. Приложение готово к сдаче и может служить основой для дальнейшего развития (микросервисы, контейнеризация).